

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

An AUTONOMOUS Institution

Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade

Recognized by UGC under Sections 2(f) and 12(B) of UGC Act, 1956

Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.

Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING



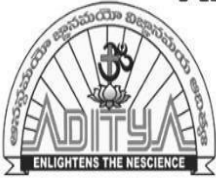
POWER SYSTEMS SIMULATION LAB

(COURSE CODE: 231EE5L01)

(V SEMESTER)

LAB MANUAL

ACET R23 REGULATION



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

An AUTONOMOUS Institution

Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade

Recognized by UGC under Sections 2(f) and 12(B) of UGC Act, 1956

Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.

Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

Department of Electrical and Electronics Engineering

Name:

Reg. No:

--	--	--	--	--	--	--	--	--	--

*Certified that this is the Bonafide record of
practical work done by*

Mr./Ms.....

a student of..... with Reg No.....in

the Laboratory during the year.....

No. of Practical's Conducted

No. of Practical's Attended

Signature of Faculty In-Charge

Signature of Head of the Department

Submitted for the practical examination held on



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

An AUTONOMOUS Institution

Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade

Recognized by UGC under Sections 2(f) and 12(B) of UGC Act, 1956

Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.

Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

Day – To – Day Evaluation

Student Copy to Be Maintained in The Record:

Roll No:

Academic Year:

Name of the Student:

Semester:

Name of the Lab:

Course Code:

S.No	Name of The Experiment	P1: Experiments Performed Daily & Submission of Record (10 M)	Faculty Sign
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			
Total (P1)			
Average (P1) – 10 Marks			

INDEX

[illegible]

Semester: VII	POWER SYSTEMS SIMULATION LAB	L	T	P	C
Code: 231EE7S01		0	0	3	2

Pre-requisite: Power System Analysis, Operation and Control

Course Objectives:

- Understand the formation of Y-Bus and Z-Bus matrices for power system network modeling.
- Apply various load flow analysis methods including Gauss-Seidel, Newton-Raphson, and Fast Decoupled techniques.
- Analyze symmetrical and unsymmetrical faults using Z-Bus and sequence components.

Course Outcomes:

After the completion of the course the student should be able to:

CO1	Distinguish between different load flow methods.
CO2	Building and analyzing Y-bus & Z-bus algorithms.
CO3	Practice and analyze the symmetrical & unsymmetrical faults.
CO4	Recognize the importance of Load flow control and Economic load dispatch.
CO5	Recognize the importance of Economic load dispatch and transient stability analysis.

CO-PO-PSO Mapping

CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	2	2		2	2			2				2	2
CO2	3	2	2		2	1			1				2	2
CO3	3	2	2		2	2			2				2	2
CO4	3	2	2		2	1			2				2	2
CO5	3	2	2		2	1			2				2	1

List of Experiments

Any 10 of the following experiments are to be conducted

1. Formation of Y- Bus by Direct-Inspection Method.
2. Load Flow Solution Using Gauss Siedel Method
3. Load Flow Solution Using Newton Raphson Method
4. Load Flow Solution Using Decoupled Method
5. Load Flow Solution Using Fast Decoupled Method
6. Formation of Z-Bus by Z-bus building algorithm
7. Symmetrical Fault analysis using Z-bus
8. Unsymmetrical Fault analysis using Z-bus
9. Economic Load Dispatch with & without transmission losses
10. Transient Stability Analysis Using Point By Point Method
11. Load Frequency Control of Single Area Control & Two Area Control system with and without controllers.

FORMATION OF Y-BUS BY DIRECT- INSPECTION METHOD

Program

```
clc;
clear;
close all;
n = input('Enter number of buses: ');
l = input('Enter number of lines: ');
s = input('Enter 1 for Impedance input or 2 for Admittance input: ');
ybus = zeros(n,n);
y = zeros(n,n);
lc = zeros(n,n);
for i = 1:l
    a = input('Starting bus: ');
    b = input('Ending bus: ');
    t = input('Impedance/Admittance of line: ');
    lca = input('Line charging admittance: ');
    if s == 1
        y(a,b) = 1/t;
    else
        y(a,b) = t;
    end
    y(b,a) = y(a,b);
    lc(a,b) = lca;
    lc(b,a) = lca;
end
for i = 1:n
    for j = 1:n
        if i == j
            for k = 1:n
                ybus(i,i) = ybus(i,i) + y(i,k) + lc(i,k)/2;
            end
        else
            ybus(i,j) = -y(i,j);
        end
    end
end
end

disp('Y-Bus Matrix is:')
disp(ybus)
```

1. FORMATION OF Y-BUS BY DIRECT- INSPECTION METHOD

Aim:- To develop a program to obtain Y-bus matrix for the given networks by the method of Inspection.

Simulation Required: MATLAB Software

Formation of Y-Bus Matrix

$$\text{Generalized [Y-Bus]} = \begin{bmatrix} Y_{ii} & Y_{ij} \\ Y_{ji} & Y_{jj} \end{bmatrix}$$

Each admittance Y_{ii} ($i=1,2,\dots,n$) is called the self admittance or driving point admittance of bus I and equals the sum of all admittances terminating on the particular bus.

Each off-diagonal term Y_{ij} ($i,j=1,2,\dots,n$; j is the transfer admittance between buses I and j , n =total number of buses. Further, $Y_{ij} = Y_{ji}$)

Simulation:

In this exercise matrix, Z-Bus for the system is developed by first forming the Ybus and then Inverting it to get the Z-Bus matrix. The generator and transformer impedances are taken into account. Y_{bus} is a sparse matrix, Z-Bus is a full matrix, i.e., zero elements of Ybus become non-zero values in the corresponding Z-Bus elements. The bus impedance matrix is most useful for short circuit studies

Algorithm

Step (1): Initialize [Y-Bus] matrix that is replace all entries by zero.

$Y_{ij} = Y_{ij} - y_{ij} = Y_{ji}$ = off diagonal element

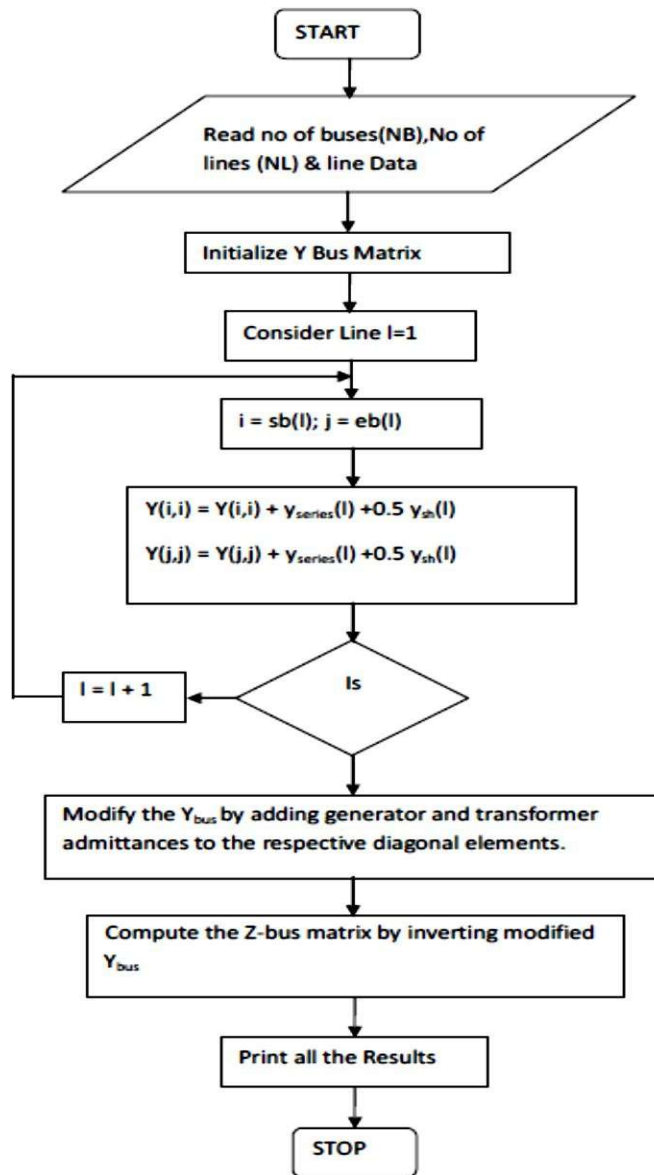
Step (2): Compute $Y_{ii} = \sum_{j=1}^n y_{ij}$ = diagonal element.

Step (3): Modify the Ybus matrix by adding the transformer and the generator admittances to the respective diagonal elements of Y- bus matrix.

Step (4): Compute the Z-Bus matrix by inverting the modified Ybus matrix.

Output

The [Y-Bus] matrix is formed by inspection method for a four bus system. The line data and is given below.





Line Data

Line Number	SB	EB	Series Impedance (p.u)	Line charging Admittance (p.u)
1	1	2	$0.10 + j0.40$	$j0.015$
2	2	3	$0.15 + j0.60$	$j0.020$
3	3	4	$0.18 + j0.55$	$j0.018$
4	4	1	$0.10 + j0.35$	$j0.012$
5	4	2	$0.25 + j0.20$	$j0.030$

Manual Calculation

Result

Viva Questions:

1. How do you calculate Ybus?
2. Why Ybus is used for load flow analysis?
3. How is Ybus formed?
4. Why do we need Ybus?



LOAD FLOW SOLUTION USING GAUSS-SEIDEL METHOD

Program

```

clc;
clear;
close all;
% Number of buses
n = 4;
% Initial voltage values (Slack bus voltage fixed)
V = [1.04 1 1 1];
% Bus Admittance Matrix (Y-bus)
Y = [ 3-j*9    -2+j*6    -1+j*3    0;
      -2+j*6    3.666-j*11  -0.666+j*2  -1+j*3;
      -1+j*3    -0.666+j*2    3.666-j*11  -2+j*6;
      0        -1+j*3    -2+j*6    3-j*9 ];
% Specified real and reactive powers
% Slack bus values are not used
P = [0 0.5 -1 0.3];
Q = [0 -0.2 0.5 -0.1];
% Convergence tolerance
tolerance = 1e-5;
% Maximum iterations
max_iter = 100;
% Initialization
iter = 0;
diff = 1;
fprintf('GAUSS-SEIDEL LOAD FLOW ANALYSIS\n');
while (diff > tolerance && iter < max_iter)
    V_prev = V;
    % Update load bus voltages
    for i = 2:n
        sumYV = 0;
        for k = 1:n
            if (k ~= i)
                sumYV = sumYV + Y(i,k) * V(k);
            end
        end
        % Gauss-Seidel voltage update equation
        V(i) = (1 / Y(i,i)) * ...
            (((P(i) - j*Q(i)) / conj(V(i))) - sumYV);
    end
    % Compute maximum voltage magnitude difference
    diff = max(abs(abs(V) - abs(V_prev)));
    iter = iter + 1;
    % Display iteration results
    fprintf('\nIteration %d\n', iter);
    for m = 1:n
        fprintf('Bus %d Voltage = %.5f + j%.5f |V| = %.5f\n', ...
            m, real(V(m)), imag(V(m)), abs(V(m)));
    end
end
fprintf('\n-----\n');
fprintf('Load Flow Converged in %d iterations\n', iter);
fprintf('-----\n');
% Final Results
disp('Final Bus Voltages:');
disp(V.);

```

2. LOAD FLOW SOLUTION USING GAUSS-SEIDEL METHOD

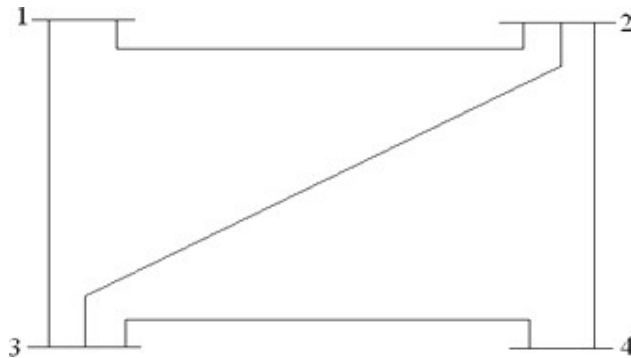
Aim:- To find load flow solution of the given power system using Gauss-Seidel method theoretically for one iteration and obtain full solution using MATLAB.

Software Required: MATLAB Software

Problem

For the sample power system shown below, the generators are connected at all the four buses, while loads are at buses 2 and 3. Values of real and reactive powers are listed in the table. All buses other than the slack are PQ type. Assuming a flat voltage start, find the voltages and bus angles at the three buses at the end of first GS iteration.

Input Data



Simple 4-bus power system

Bus	P_i , pu	Q_i , pu	V_i , pu	Remarks
1	-	-	1.04 $\angle 0^\circ$	Slack bus
2	0.5	-0.2	-	PQ bus
3	-1.0	0.5	-	PQ bus
4	0.3	-0.1	-	PQ bus

For the above system, the bus admittance matrix is

$$Y_{BUS} = \begin{bmatrix} +3-j9 & -2.00+j6 & -1.00+j3 & 0 \\ -2+j6 & +3.666-j11 & -0.666+j2 & -1+j3 \\ -1+j3 & -0.666+j2 & +3.666-j11 & -2+j6 \\ 0 & -1.00+j3 & -2.00+j6 & +3-j9 \end{bmatrix}$$

Output

Procedure

1. Open MATLAB
2. Open new M-file
3. Type the program, after save file **.m** format.
4. Compile and Run the program
5. Observe the output Results in Command window
6. Take printouts of that Result.

Result

Viva Questions:

1. Why we use Gauss Seidel method in load flow analysis?
2. What is the purpose of load flow studies?
3. Which matrix is used for load flow studies?
4. What are the works involved in a load flow study?

LOAD FLOW SOLUTION USING NEWTON RAPHSON METHOD

Program

```

clc;
clear;
close all;
% BUS DATA
% [Bus No Pg Qg Pl Ql]
gbus = [1 2.0 1.0 0.0 0.0;
        2 0.0 0.0 0.5 1.0;
        3 1.5 0.6 0.0 0.0];
% Y-BUS MATRIX
ybus = [ 5.882-j*23.528 -2.941+j*11.764 -2.941+j*11.764;
        -2.941+j*11.764 5.882-j*23.528 -2.941+j*11.764;
        -2.941+j*11.764 -2.941+j*11.764 5.882-j*23.528];
% TOLERANCE
tol = 0.001;
% INITIAL VOLTAGES
V1 = 1.04 + j*0;
V2 = 1.00 + j*0;
V3 = 1.04 + j*0;
% Voltage magnitudes
Vm1 = abs(V1);
Vm2 = abs(V2);
Vm3 = abs(V3);
% Voltage angles
del1 = angle(V1);
del2 = angle(V2);
del3 = angle(V3);
% POWER CALCULATIONS
% Real Power at Bus-2
P2 = Vm2*Vm1*abs(ybus(2,1))* ...
    cos(angle(ybus(2,1)) + del1 - del2) ...
    + Vm2^2*abs(ybus(2,2))* ...
    cos(angle(ybus(2,2))) ...
    + Vm2*Vm3*abs(ybus(2,3))* ...
    cos(angle(ybus(2,3)) + del3 - del2);
% Reactive Power at Bus-2
Q2 = -Vm2*Vm1*abs(ybus(2,1))* ...
    sin(angle(ybus(2,1)) + del1 - del2) ...
    -Vm2^2*abs(ybus(2,2))* ...
    sin(angle(ybus(2,2))) ...
    -Vm2*Vm3*abs(ybus(2,3))* ...
    sin(angle(ybus(2,3)) + del3 - del2);

% Real Power at Bus-3
P3 = Vm3*Vm1*abs(ybus(3,1))* ...
    cos(angle(ybus(3,1)) + del1 - del3) ...
    + Vm3^2*abs(ybus(3,3))* ...
    cos(angle(ybus(3,3))) ...
    + Vm2*Vm3*abs(ybus(3,2))* ...
    cos(angle(ybus(3,2)) + del2 - del3);

```

3. LOAD FLOW SOLUTION USING NEWTON RAPHSON METHOD

Aim:- To develop in MATLAB to find load flow studies using Newton Raphson and fast decoupled load flow method.

Software Required: MATLAB Software

Circuit Diagram

STEP 1: Assume a flat profile $1+j0$ for all buses except the slack bus in the Specified voltage and it is not modified in any iteration.

STEP 2: Assume a suitable value of ϵ called convergence criterion. Hence ϵ is A specified change in the residue that is used to compare the actual residue that is used to compare the actual residue at the end of each iteration.

STEP 3: Set the iteration count $K=0$ and assumed voltage profile of the buses are denoted as $V_1^0, V_2^0, \dots, V^0$.

STEP 4: Set the bus count $p=1$

STEP 5: Check for slack bus. If it is a slack bus then go to step 13. Otherwise go to next step.

STEP 6: Calculate the real & reactive power of bus p using the following equation,

$$P_{pk} = \sum_{q=1}^n \{ e_p (e_q^k G_{pq}^k + f_q^k B_{pq}^k) + f_p (f_q^k G_{pq}^k - e_q^k B_{pq}^k) \}$$

$$Q_{pk} = \sum_{q=1}^n \{ f_p (e_q^k G_{pq}^k + f_q^k B_{pq}^k) + e_p (f_q^k G_{pq}^k - e_q^k B_{pq}^k) \}$$

STEP 7: Calculate the change in real

$$\text{power, } \Delta P_p^k = P_{p \text{ spec}} - P_{pk}^k$$

Where, P_p

$P_{p \text{ spec}}$ = specified real power of bus p .

STEP 8: Check for generator bus. If it is a generator bus gob to next step otherwise go to step 12.

STEP 9: Check for generator bus. If it is a generator reactive power limit Violation of generator buses. for this compare the calculated. Reactive power Q with specified limits. If the limits are violated go to step 11. Otherwise go to next step.

STEP 10: If the calculated reactive power is within the soecified limit then consider this bus as generator bus. Now calculate the voltage residue using the equation

$$|V_p^k| - |V_{p \text{ spec}}| = |V_p^k| - |V_p^{\text{spec}}|$$

STEP 11: If the reactive power limit violated the treat this bus as a load bus. Now the specified reactive power for this bus will correspond to Limit violated

STEP 12: Calculate the change in reactive power for load bus change in reactive power,

$$\Delta Q_k = |Q_{p \text{ spec}}| - Q_{pk}^k$$

% POWER MISMATCHES

```
delP2 = gbus(2,4) - gbus(2,2) - P2;
delP3 = gbus(3,4) - gbus(3,2) - P3;
delQ2 = gbus(2,5) - gbus(2,3) - Q2;
```

% JACOBIAN MATRIX

```
J = zeros(3,3);
J(1,1) = Vm2*Vm1*abs(ybus(2,1))* ...
    sin(angle(ybus(2,1)) + del1 - del2) ...
    + Vm2*Vm3*abs(ybus(2,3))* ...
    sin(angle(ybus(2,3)) + del3 - del2);
J(1,2) = -Vm2*Vm3*abs(ybus(2,3))* ...
    sin(angle(ybus(2,3)) + del3 - del2);
J(1,3) = Vm1*abs(ybus(2,1))* ...
    cos(angle(ybus(2,1)) + del1 - del2) ...
    + 2*Vm2*abs(ybus(2,2))* ...
    cos(angle(ybus(2,2))) ...
    + Vm3*abs(ybus(2,3))* ...
    cos(angle(ybus(2,3)) + del3 - del2);
J(2,1) = -Vm3*Vm2*abs(ybus(3,2))* ...
    sin(angle(ybus(3,2)) + del2 - del3);
J(2,2) = Vm3*Vm1*abs(ybus(3,1))* ...
    sin(angle(ybus(3,1)) + del1 - del3) ...
    + Vm3*Vm2*abs(ybus(3,2))* ...
    sin(angle(ybus(3,2)) + del2 - del3);
J(2,3) = Vm3*abs(ybus(3,2))* ...
    cos(angle(ybus(3,2)) + del2 - del3);
J(3,1) = Vm2*Vm1*abs(ybus(2,1))* ...
    cos(angle(ybus(2,1)) + del1 - del2) ...
    - Vm2*Vm3*abs(ybus(2,3))* ...
    cos(angle(ybus(2,3)) + del3 - del2);
J(3,2) = Vm2*Vm3*abs(ybus(2,3))* ...
    cos(angle(ybus(2,3)) + del3 - del2);
J(3,3) = -Vm2*abs(ybus(2,1))* ...
    sin(angle(ybus(2,1)) + del1 - del2) ...
    -2*Vm2*abs(ybus(2,2))* ...
    sin(angle(ybus(2,2))) ...
    -Vm3*abs(ybus(2,3))* ...
    sin(angle(ybus(2,3)) + del3 - del2);
```

% DISPLAY JACOBIAN

```
disp('Jacobian Matrix J = ');
disp(J);
```

% STATE VECTOR

```
A = [del2;
    del3;
    Vm2];
```

% Mismatch vector

```
delA = [delP2;
    delP3;
    delQ2];
```

STEP 13: Repeat the step 5 to 12 until all residues are calculated for increment the bus count n. by 1 to 5 steps until the bus count is n.

STEP 14: Determine the largest of the absolute value of the residue (i.e.) Find the largest value among ΔP^k , ΔQ^k or $|\Delta V|^2$

STEP 15: Compare ΔE and E , if $\Delta E < E$ then go to step 20. If $\Delta E > E$ go to next Step.

STEP 16: Determine the element, the load flow equation using k^{th} iteration Value.

STEP 17: Calculate the increment in real and reactive part of voltage Δe^k and Δf^k by solving the matrix equation $B=JC$.

STEP 18: Calculate the new bus voltage.

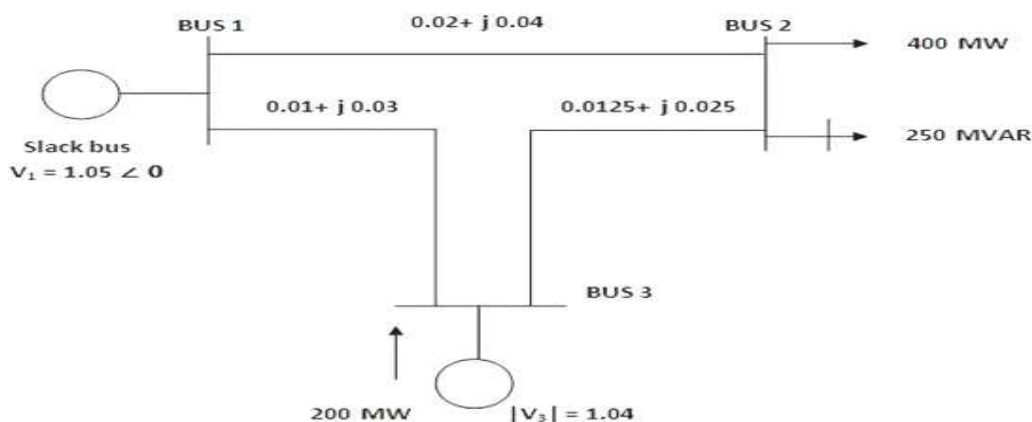
STEP 19: Advance the iteration count $k=k+1$ and go to step4.

STEP 20: Calculate the line flows.

Problems

Figure shows the one-line diagram of a three-bus power system with generators at buses 1 & 3. The magnitude of voltage at bus1 is adjusted to 1.05 per unit. The magnitude of voltage at bus 3 is fixed at 1.04pu with a real power generation of 200 MW. A load consists of 400 MW and 250 MVAR is taken from bus2. Line impedances are marked in per unit on a 100MVA base and the line charging susceptances are neglected.

- Obtain the load flow solution by Newton-Raphson method.
- Write and execute a MATLAB program and also verify the output with the manual calculation results.



```
% CORRECTION VECTOR
corr = inv(J) * delA;
% Updated values
A_new = A + corr;
% RESULTS
fprintf('\nUpdated Values:\n');
fprintf('Delta2 = %f radians\n', A_new(1));
fprintf('Delta3 = %f radians\n', A_new(2));
fprintf('V2 Magnitude = %f pu\n', A_new(3));
% CONVERGENCE CHECK
if max(abs(corr)) < tol
    fprintf('\nLoad flow converged.\n');
else
    fprintf('\nLoad flow not yet converged.\n');
end
```

Output

Procedure

1. Open MATLAB
2. Open new M-file
3. Type the program, after save file **.m** format.
4. Compile and Run the program
5. Observe the output Results in Command window
6. Take printouts of that Result.

Result



Viva Questions:

1. What is meant by newton raphson method?
2. What is Y bus in power system?
3. What is Newton Raphson method in power system?
4. What is generator bus?
5. What is fast decoupled flow method?



LOAD FLOW SOLUTION USING DECOUPLED METHOD

Program

```

% Decoupled Load Flow (FDLF) Method
clc;
clear;
close all;
% Number of buses
nb = 3;
% Bus Admittance Matrix (Ybus)
Y = [10-20i  -5+10i  -5+10i;
     -5+10i  10-20i  -5+10i;
     -5+10i  -5+10i  10-20i];
% Imaginary part matrices
B1 = -imag(Y);
B2 = -imag(Y);
% Initial Voltage Magnitudes and Angles
V = ones(nb,1);      % Voltage magnitude in p.u
delta = zeros(nb,1); % Voltage angles in radians
% Specified Active and Reactive Powers
Psp = [0; -1.0; -0.8];
Qsp = [0; -0.5; -0.3];
% Tolerance and Maximum Iterations
tolerance = 1e-4;
max_iter = 20;
fprintf('Fast Decoupled Load Flow Analysis\n');
for iter = 1:max_iter
    % Initialize calculated powers
    P = zeros(nb,1);
    Q = zeros(nb,1);
    % Calculate Active and Reactive Power
    for i = 1:nb
        for j = 1:nb
            G = real(Y(i,j));
            B = imag(Y(i,j));
            P(i) = P(i) + V(i)*V(j)* ...
                (G*cos(delta(i)-delta(j)) + ...
                B*sin(delta(i)-delta(j)));
            Q(i) = Q(i) + V(i)*V(j)* ...
                (G*sin(delta(i)-delta(j)) - ...
                B*cos(delta(i)-delta(j)));
        end
    end
    % Power mismatches
    dP = Psp - P;
    dQ = Qsp - Q;
    % Remove slack bus mismatch
    dP_red = dP(2:nb);
    dQ_red = dQ(2:nb);
end

```

4. LOAD FLOW SOLUTION USING DECOUPLED METHOD

Aim:-

To perform load flow analysis of a power system using the **Fast Decoupled Load Flow (FDLF)** method and obtain:

- Bus voltages (magnitude & angle)
- Real and reactive power flows
- Line losses

Simulation Required: MATLAB Software

Theory

Load flow analysis is a fundamental study in **power system analysis** used to determine the steady-state operation of a system.

The **Fast Decoupled Load Flow (FDLF)** method is derived from the Newton-Raphson method by simplifying the Jacobian matrix based on the assumptions:

- Real power (P) is strongly dependent on voltage angle (δ)
- Reactive power (Q) is strongly dependent on voltage magnitude ($|V|$)

Decoupled Equations:

$$\Delta P = B' \Delta \delta$$

$$\Delta Q = B'' \Delta V$$

Where:

- (B') and (B'') are simplified susceptance matrices
- Iterations are done separately for P- δ and Q-V

Advantages:

- Faster computation
- Less memory requirement
- Suitable for large systems

```
% Reduced B matrices
B1_red = B1(2:nb,2:nb);
B2_red = B2(2:nb,2:nb);
% Angle correction
d_delta = B1_red \ dP_red;
% Voltage magnitude correction
dV = B2_red \ dQ_red;
% Update values
delta(2:nb) = delta(2:nb) + d_delta;
V(2:nb) = V(2:nb) + dV;
% Check convergence
if max(abs([dP_red; dQ_red])) < tolerance
    fprintf('\nConverged in %d iterations\n', iter);
    break;
end
end
% Display Results
fprintf('\nBus Voltage Magnitudes (p.u):\n');
disp(V);
fprintf('Bus Voltage Angles (radians):\n');
disp(delta);
fprintf('Bus Voltage Angles (degrees):\n');
disp(delta * 180/pi);
```

Software Requirement

- MATLAB / PSAT / ETAP / PowerWorld

Simulator System Data (Example: 3-Bus System)

Bus	Type	V (pu)	δ (deg)	Pg (pu)	Qg (pu)	PL (pu)	QL (pu)
1	Slack	1.05	0	-	-	0	0
2	PV	1.02	-	0.5	-	0.2	0.1
3	PQ	-	-	0	0	0.6	0.3

Procedure**Step 1: Input Data**

- Enter bus data (Slack, PV, PQ)
- Enter line data (impedance, admittance)

Step 2: Form Y-Bus Matrix

- Construct the admittance matrix

Step 3: Initialize Values

- Voltage magnitude = 1.0 pu (for PQ buses)
- Angle = 0° (except slack)

Step 4: Compute Power Mismatch

$$\Delta P_i = P_i^{\text{specified}} - P_i^{\text{calculated}}$$

$$\Delta Q_i = Q_i^{\text{specified}} - Q_i^{\text{calculated}}$$

Step 5: Solve Decoupled Equations

- Solve ($\Delta P = B' \Delta \delta$)
- Update δ
- Solve ($\Delta Q = B'' \Delta V$)
- Update voltage magnitude

Output

Step 6: Iteration

- Repeat until mismatch is within tolerance (e.g., 0.001)

Step 7: Compute Results

- Bus voltages
- Line flows
- Power losses

Result:

Viva Questions

1. What is load flow analysis?
2. Difference between Newton-Raphson and FDLF method?
3. What are Slack, PV, and PQ buses?
4. Why is FDLF faster?
5. What are the assumptions in decoupled load flow?

LOAD FLOW SOLUTION USING FAST DECOUPLED METHOD

Program

```

% Fast Decoupled Load Flow (FDLF)

clc;
clear;
close all;
% Number of buses
nb = 3;
% Bus Admittance Matrix (complex form)
Y = [10-20i  -5+10i  -5+10i
      -5+10i  10-20i  -5+10i
      -5+10i  -5+10i  10-20i];
% Susceptance matrix
B = -imag(Y);
% Initial voltages and angles
V = ones(nb,1);
delta = zeros(nb,1);
% Specified powers
Psp = [0; -0.5; -0.6];
Qsp = [0; -0.2; -0.3];
% Convergence parameters
tolerance = 1e-4;
max_iter = 20;
for iter = 1:max_iter
    % Initialize P and Q
    P = zeros(nb,1);
    Q = zeros(nb,1);
    % Power calculations
    for i = 1:nb
        for j = 1:nb
            G = real(Y(i,j));
            Bij = imag(Y(i,j));
            P(i) = P(i) + V(i)*V(j)* ...
                (G*cos(delta(i)-delta(j)) + ...
                Bij*sin(delta(i)-delta(j)));
            Q(i) = Q(i) + V(i)*V(j)* ...
                (G*sin(delta(i)-delta(j)) - ...
                Bij*cos(delta(i)-delta(j)));
        end
    end
end

```

5. LOAD FLOW SOLUTION USING FAST DECOUPLED METHOD

Aim:

To perform load flow analysis of a power system network using the **Fast Decoupled Load Flow (FDLF) Method** and determine:

- Bus voltages
- Voltage angles
- Real and reactive power flows

Simulation Required: MATLAB Software

Objective

- Understand the concept of load flow studies
- Implement the fast decoupled algorithm using simulation software (like MATLAB / PSAT / PowerWorld)
- Analyze system performance under steady-state conditions

Theory

Load flow analysis is a key study in **Power System Engineering** used to determine the steady- state operation of power systems.

The **Fast Decoupled Load Flow Method** is derived from the **Newton-Raphson Method**, but simplifies calculations by decoupling:

- Active power (P) with voltage angle (δ)
- Reactive power (Q) with voltage magnitude (V)

Basic Assumptions

- Line resistance is negligible compared to reactance
- Voltage angle differences are small
- System is heavily inductive

```
end
end
% Power mismatches
dP = Psp - P;
dQ = Qsp - Q;
% Remove slack bus
dP_red = dP(2:nb);
dQ_red = dQ(2:nb);
% Reduced B matrix
Bred = B(2:nb,2:nb);
% Corrections
d_delta = Bred \ dP_red;
dV = Bred \ dQ_red;

% Update values
delta(2:nb) = delta(2:nb) + d_delta;
V(2:nb) = V(2:nb) + dV;
% Convergence check

if max(abs([dP_red; dQ_red])) < tolerance
    fprintf('Converged in %d iterations\n', iter);
    break;
end
end

% Display results
disp('Voltage Magnitudes:')
disp(V)
disp('Voltage Angles (radians):')
disp(delta)
disp('Voltage Angles (degrees):')
disp(delta * 180/pi)
```

Equations Used

The load flow equations are simplified

into:

$$\Delta P = B' \Delta \delta$$

$$\Delta Q = B'' \Delta V$$

Where:

- (B'), (B'') = Susceptance matrices
- (ΔP), (ΔQ) = Power mismatches
- ($\Delta \delta$), (ΔV) = Corrections

Algorithm (Step-by-Step)

1. **Input system data**
 - Bus data (Slack, PV, PQ buses)
 - Line data (impedance, admittance)
2. **Form Y-bus matrix**
3. **Initialize values**
 - Voltage magnitude ($V = 1.0$) p.u.
 - Voltage angle ($\delta = 0$)
4. **Compute power mismatches**
 - (ΔP), (ΔQ)
5. **Form B' and B'' matrices**
6. **Solve equations**
 - Update (δ) using (ΔP)
 - Update (V) using (ΔQ)
7. **Check convergence**
 - If mismatch < tolerance → Stop
 - Else → Repeat steps

Simulation Procedure (MATLAB Example)

1. Open **MATLAB**
2. Enter bus and line data
3. Write/load FDLF program
4. Run simulation
5. Observe:

Output

Advantages of Fast Decoupled Method

- Faster than full Newton-Raphson
- Requires less memory
- Suitable for large systems

Disadvantages

- Less accurate for:
 - High R/X ratio systems
 - Poorly conditioned networks

Applications

- Power system planning
- Voltage stability studies
- Operation of transmission networks

Result

Viva Questions

1. What is load flow analysis?
2. Difference between Newton-Raphson and FDLF method?
3. What are Slack, PV, and PQ buses?
4. Why is FDLF faster?

FORMATION OF Z-BUS BY Z-BUS BUILDING ALGORITHM

Program

```

clc;
clear all;
nbus=input('Enter the total no. of buses excluding reference bus');
zbus=zeros(nbus,nbus);
t=1;
while t==1
    s=menu('Specify the case number','Connect a new bus to reference bus','connect an old bus to new bus','Connect two old buses','Connect an old bus to reference bus','Quit');
    switch(s);
    case {1}
        zb=input('Enter the Zbus value');
        zbus=zb;
    case {2}
        n=input('Enter the 2nd bus no.:');
        zb=input('Enter the reactance value:');
        for i=1:n
            if i==n
                zbus(n,n)=zbus(n-1,n-1)+zb;
            else
                zbus(i,n)=zbus(i,n-1);
                zbus(n,i)=zbus(n-1,i);
            end
        end
        case {3}
            l=input('Enter the existing bus number 1:');
            n=input('Enter the existing bus number 2:');
            zb=input('Enter the reactance value:');
            n=n+1;
            for i=1:n
                if i==n
                    zbus(n,n)=zbus(l,l)+zb+zbus(n-1,n-1)-(2*zbus(l,n-1));
                    zbus(n,n);
                else
                    zbus(i,n)=zbus(l,i)-zbus(i,n-1);
                    zbus(i,n); zbus(n,i)=zbus(i,n);
                end
            end
            for i=1:nbus
                for j=1:nbus
                    if i==j
                        zbus(i,j)=zbus(i,j)-(zbus(i,n)*zbus(n,j)/zbus(n,n));
                        zbus(j,i)=zbus(i,j);
                        zbus(i,j)=zbus(j,i);
                    else
                        zbus(i,j)=zbus(i,j)-(zbus(i,n)*zbus(n,j)/zbus(n,n));
                    end
                end
            end
            zbus(i,n)=0;
            zbus(n,i)=0;
        case {4}

```

6. FORMATION OF Z-BUS BY Z-BUS BUILDING ALGORITHM

Aim:- To develop a program to obtain the Z bus matrix for the given network by the method of bus building algorithm.

Simulation Required: MATLAB Software

Formation of Z-Bus Matrix

Z-Bus matrix is an important matrix used in different kinds of power system studies such as short circuit study, load flow study, etc.

In short circuit analysis, the generator and transformer impedances must be taken into account.

In contingency analysis, the shunt elements are neglected while forming the Z-Bus matrix, which is used to compute the outage distribution factors.

This can be easily obtained by inverting the Ybus formed by inspection method or by analytical method. Taking inverse of the Ybus for large systems is time consuming; Moreover, modification in the system requires the whole process to be repeated to reflect the changes in the system. In such cases, the Z-Bus is computed by Z-Bus building algorithm.

Algorithm

Step 1: Start the program.

Step 2: Read the number of buses, starting bus and ending

bus. Step 3: Initialize the Z Bus matrix.

Step 4: Form the Z –Bus matrix as follows

Case 1: When a new bus of impedance Z_b is connected to reference bus $Z_{bus,new} = [Z_b]$

Case 2: Adding new bus p to existing bus q

$$Z_{bus,new} = \begin{bmatrix} Z_{orig} & Z_{1q} \\ & Z_{2q} \\ & \dots \\ Z_{q1} & Z_{q2} & \dots & Z_{qq} + Z_b \end{bmatrix}$$

```
n=input('Enter the bus number:');
zb=input('Enter reactance value:');
n=n+1;
for i=1:n
    if i==n
        zbus(n,n)=zbus(n-1,n-1)+zb;
    else
        zbus(i,n)=zbus(i,n-1);
        zbus(n,i)=zbus(n-1,i);
    end
end
for i=1:n
    for j=1:n
        if i==j
            zbus(i,j)=zbus(i,j)-(zbus(i,n)*zbus(n,j)/zbus(n,n));
            zbus(j,i)=zbus(i,j);
            zbus(i,j)=zbus(j,i);
        else
            zbus(i,j)=zbus(i,j)-(zbus(i,n)*zbus(n,j)/zbus(n,n));
        end
    end
end
case {5}
disp('End the program');
choice=menu('Would you like to print program or end program','print','end');
if choice==1
    zbus
else
    t=0;
end
end
```

Case 3: Adding impedance from an existing bus to reference bus.

$$Z_{jk,act} = Z_{jk} - \frac{Z_{j(n+1)} * Z_{(n+1)k}}{Z_{(n+1)(n+1)}}$$

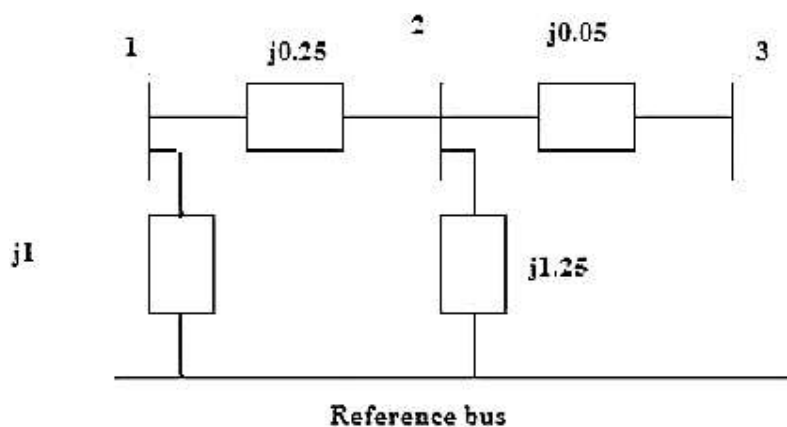
Case 4: Adding Z_b between two existing buses h and q

$$Z_{bus,new} = \begin{bmatrix} Z_{orig} & Z_{1h} - Z_{1q} \\ & Z_{2h} - Z_{2q} \\ & Z_{h1} - Z_{q1} \quad Z_{h2} - Z_{q2} \dots Z_{(n+1)(n+1)} \end{bmatrix}$$

Case 5: Print the Z –bus matrix.

Problem

Find the bus impedance matrix for the given network



Formation of Z Bus by Bus Building Algorithm

Output

Result

Viva Questions:

1. What is Z bus building algorithm?
2. Why Z bus is not used in load flow analysis?
3. What are the merits of Z bus building algorithm?
4. Why is Z bus full matrix?



SYMMETRICAL FAULT ANALYSIS USING Z-BUS

Program

```
clc;

clear;

% Faulted bus

k = 2;

% Modified Y-Bus Matrix

Y = [15 -5 -5;
     -5 15 -5;
     -5 -5 15];

% Z-Bus Matrix

Z = inv(Y);

% Pre-fault voltage

V_prefault = 1;

% Fault current

If = V_prefault / Z(k,k);

% Bus voltages during fault

V = ones(3,1) - Z(:,k) * If;

disp('Z-Bus Matrix:');

disp(Z);

disp('Fault Current:');

disp(If);

disp('Bus Voltages:');

disp(V);
```

7.SYMMETRICAL FAULT ANALYSIS USING Z-BUS

Aim:

To perform symmetrical (3-phase) fault analysis on a power system using the **bus impedance matrix (Z-Bus)** and determine fault current and bus voltages during the fault.

Simulation Required: MATLAB Software

Theory

A **symmetrical fault** (balanced 3-phase fault) affects all three phases equally. It is the most severe fault and is analyzed using **positive sequence network only**.

Z-Bus Matrix

The **bus impedance matrix (Z-Bus)** relates bus voltages and injected

currents:

$$V = [Z_{Bus}] \cdot [I]$$

Where:

- (V) = Bus voltage vector
- (I) = Bus current injection vector
- (Z_{bus}) = Bus impedance matrix

Fault Current Calculation

For a 3-phase fault at bus k :

$$I_f = \frac{V_k^0}{Z_{kk} + Z_f}$$

Where:

where:

- I_f = Fault current at bus k (p.u. or A)
- V_k^0 = Prefault voltage at the faulted bus (usually $1\angle 0^\circ$ p.u.)
- Z_{kk} = Driving-point (self) impedance of bus k obtained from the Z-Bus matrix
- Z_f = Fault impedance (p.u. or Ω)

Bus Voltages During Fault

$$V_i = V_i^0 - Z_{ik} I_f$$

Output

where:

- V_i = Bus voltage at bus i during the fault
- V_i^0 = Prefault voltage at bus i (usually $1\angle 0^\circ$ p.u.)
- Z_{ik} = Transfer impedance between buses i and k
- I_f = Fault current at the faulted bus

Apparatus / Software Required

- MATLAB / Simulink (or)
- ETAP / PSCAD / PowerWorld Simulator
- Computer system

System Data (Example)

(You can modify based on your lab)

- Number of buses: 3 or 4
- Generator reactance: ($X_g = j0.2$)
- Line reactances:
 - Line 1–2: ($j0.4$)
 - Line 2–3: ($j0.3$)
- Base MVA: 100 MVA

Procedure

Step 1: Form Y-Bus Matrix

- Calculate admittances:

$$Y_{ij} = \frac{1}{Z_{ij}}$$

- Construct the Y-Bus matrix.

Step 2: Compute Z-Bus Matrix

- Invert Y-Bus:

$$Z_{Bus} = Y_{Bus}^{-1}$$

Step 3: Select Fault Bus

- Assume fault at bus k
- Assume:

- Pre-fault voltage = 1 p.u.
- Fault impedance ($Z_f = 0$)

Step 4: Calculate Fault Current

$$I_f = \frac{1}{Z_{kk}}$$

Step 5: Calculate Bus Voltages

$$V_i = 1 - Z_{ik} I_f$$

Result



Viva Questions

1. What is a symmetrical fault?
2. Why is only positive sequence network used?
3. What is Z-Bus?
4. Difference between Y-Bus and Z-Bus?
5. What happens if fault impedance is non-zero?

UNSYMMETRICAL FAULT ANALYSIS USING Z-BUS

Program:

```
clc;

clear;

% Number of buses

n = 3;

% Example Zbus matrix (pu)

Zbus = [0.2  0.05 0.04;

        0.05 0.25 0.06;

        0.04 0.06 0.3];

% Faulted bus

k = 2;

% Prefault voltage

V = 1;

% Sequence impedances

Z1 = Zbus(k,k);

Z2 = Zbus(k,k);

Z0 = Zbus(k,k);

% Fault impedance

Zf = 0;

% SLG Fault Current

If = (3 * V) / (Z1 + Z2 + Z0 + 3*Zf);

disp('Fault Current (SLG):');

disp(If);

% Bus voltages during fault

Vf = V - Zbus(:,k) * If;

disp('Bus Voltages during fault:');

disp(Vf);
```

8.UNSYMMETRICAL FAULT ANALYSIS USING Z-BUS

Aim

To analyze unsymmetrical faults (SLG, LL, and LLG faults) in a power system using the **Z-Bus method** and simulate the results using **MATLAB**.

Simulation Required: MATLAB Software

Objectives

- To understand the use of **Z-Bus matrix** in fault analysis
- To compute fault currents and bus voltages under unsymmetrical faults
- To implement the analysis using MATLAB

Theory

Unsymmetrical Faults

Unsymmetrical faults are faults where all three phases are not equally affected. Types include:

1. **Single Line-to-Ground (SLG) fault**
2. **Line-to-Line (LL) fault**
3. **Double Line-to-Ground (LLG) fault**

Z-Bus Method

- The **bus impedance matrix (Z-Bus)** is used to determine fault currents.
- It relates bus voltages and currents:

$$[V] = [Z_{Bus}][I]$$

Sequence Networks

Unsymmetrical fault analysis uses:

- Positive sequence network (Z_1)
- Negative sequence network (Z_2)
- Zero sequence network (Z_0)

Fault Current Formulas

Output

(i) SLG Fault

$$I_f = \frac{3V_k^0}{Z_{kk}^1 + Z_{kk}^2 + Z_{kk}^0 + 3Z_f}$$

(ii) LL Fault

$$I_f = \frac{\sqrt{3} V_k^0}{Z_{kk}^1 + Z_{kk}^2 + Z_f}$$

(iii) LLG Fault

$I_f = 3I_0$, where I_0 is obtained from the sequence-network equations

$$I_0 = -I_1 \frac{Z_{kk}^2}{Z_{kk}^2 + Z_{kk}^0 + 3Z_f}$$

Algorithm (Procedure)

1. Input system data (bus, line impedances)
2. Form Y-Bus matrix
3. Invert Y-Bus to get Z-Bus
4. Extract sequence impedances (Z_1, Z_2, Z_0)
5. Select fault type and location
6. Compute fault current using formulas
7. Calculate bus voltages during fault
8. Display results





Result

Viva Questions

1. What is a unsymmetrical fault?
2. What is Z-Bus matrix?
3. Why are sequence networks used?
4. Which fault is most common? (SLG)
5. Why is zero-sequence absent in LL fault?

ECONOMIC LOAD DISPATCH WITH TRANSMISSION LOSSES

Program

```
clc;

clear all;

n=2;

lamda=input('enter the values of lamda');

alpha=[0.05 0.06];

beta=[15 18];

b=[0.0012 0;0 0];

pg=zeros(n,1)

for i=1:n

    sigma(i)=b(i,:)*pg-b(i,i)*pg(i);

    pg(i)=(1-(beta(i)/lamda)-(2*sigma(i)))/(((2*alpha(i))/lamda)+2*b(i,i));

end

disp('the distribution of load shared by 2 units:'),pg

disp('the losses are'),pl=pg'*b*pg
```

9(a). ECONOMIC LOAD DISPATCH WITH TRANSMISSION LOSSES

Aim:- To understand the fundamentals of economic load dispatch and solve the problem using classical method with line losses.

Simulation Required: MATLAB Software

Algorithm

- 1) Initially choose
- 2) Assume $p_i=0$ $i=1,2,3, \dots n$.
- 3) Solve equation iteratively for P, Σ .
- 4) Calculate PL using equation.
- 5) Check if power balance is satisfied.
- 6) Increase
- 7) Repeat step3.

Question

1. Two power plants are connected together by a transmission line and load at plant
2. When 100 MW are transmitted from plant-1, the transmission loss is 12MW. The cost characteristics of two plants are

$$C_1 = 0.05p_{g1}^2 + 15p_{g1} \text{ Rs/hr}$$
$$C_2 = 0.06p_{g2}^2 + 18p_{g2} \text{ Rs/hr}$$

Find the optimum generation when $\lambda = 22$

Procedure

1. Open MATLAB
2. Open new M-file
3. Type the program, after save file **.m** format.
4. Compile and Run the program
5. Observe the output Results in Command window
6. Take printouts of that Result.

Output

Result

Viva Questions:

1. What is economic load dispatch with losses?
2. What is the condition for economic load dispatch?
3. What is the purpose of economic dispatch problem?
4. What is penalty factor in economic load dispatch?



ECONOMIC LOAD DISPATCH WITHOUT TRANSMISSION LOSSES

Program

```
clc;

clear all;

n=input('enter no of power plants:')

pd=input('power demand:')

A=[500 400 200]

B=[5.3 5.5 5.8]

G=[0.004 0.006 0.009]

for m=1:n

    x(m)=B(m)./(2*G(m));

    y(m)=1./(2*G(m));

end

L=(pd+sum(x))./sum(y);

for m=1:n

    p(m)=(L-B(m))./(2*G(m));

end

for m=1:n

    cost(m)=A(m)+(B(m)*p(m))+(G(m)*(p(m)^2));

end

cost=sum(cost)

disp('load shared by gen1'),p(1)

disp('load shared by gen2'),p(2)

disp('load shared by gen3'),p(3)

disp('total cost for generation'),cost
```

9(b).ECONOMIC LOAD DISPATCH WITHOUT TRANSMISSION LOSSES

Aim:- To understand the fundamentals of economic load dispatch and Solve the problem using classical method without line losses.

Simulation Required Matlab Software.

Algorithm

- 1) Start the program
- 2) Read the input data values
- 3) Start the iteration counter
- 4) Check the test for convergence
- 5) Increment the iterations
- 6) Calculate P, x, y ,L values
- 7) Display the above values
- 8) To find the total cost
- 9) Stop the program.
- 10) Print the value of total cost

Question

The fuel cost of 3 Thermal plants were given

below $C_1 = 500 + 5.3p_1 + 0.004p_1^2$

2

$C_2 = 400 + 5.5p_1 + 0.006p_1^2$

$C_3 = 200 + 5.8p_1 + 0.009p_1^2$

P_1, P_2, P_3 are in MW. The total demand is $P_d = 600$ MW.

Find optimal dispatch solution and total cost in Rs/hr.

Output

Procedure

1. Open MATLAB
2. Open new M-file
3. Type the program, after save file **.m** format.
4. Compile and Run the program
5. Observe the output Results in Command window
6. Take printouts of that Result.

Result



Viva Questions:

1. What is economic load dispatch?
2. What is incremental production cost
3. How incremental production cost affects the economic dispatch?
4. Write the Lagrangian equation for economic load dispatch without considering transmission line loss.
5. What could be the optimum value of incremental production cost for optimal scheduling of generators?

TRANSIENT STABILITY ANALYSIS USING POINT-BY-POINT METHOD

Program

```
clc;
clear; % System parameters
f = 50; % Frequency in Hz
H = 5; % Inertia constant
Pm = 0.8; % Mechanical input power
Pmax = 1.5; % Maximum electrical power
dt = 0.05; % Time step % Initial conditions
delta(1) = 20; % Initial rotor angle in degrees
deldelta(1) = 0; % Number of iterations
n = 20;
for i = 1:n % Electrical power
    Pe(i) = Pmax * sind(delta(i)); % Accelerating power
    Pa(i) = Pm - Pe(i); % Angular acceleration
    alpha(i) = (180 * f / H) * Pa(i); % Change in rotor angle
    deldelta(i+1) = deldelta(i) + alpha(i) * dt^2; % New rotor angle
    delta(i+1) = delta(i) + deldelta(i+1);
end
% Display results
disp('Rotor Angles:');
disp(delta); % Plot
t = 0:dt:n*dt;
figure;
plot(t, delta, 'LineWidth', 2);
xlabel('Time (sec)');
ylabel('Rotor Angle (degrees)');
title('Swing Curve using Point-by-Point Method');
grid on;
```

10. TRANSIENT STABILITY ANALYSIS USING POINT-BY-POINT METHOD

Aim

To determine the transient stability of a power system using the **point-by-point method** and to compute the rotor angle variation with time after a disturbance.

Objectives

- To understand the concept of transient stability in power systems
- To apply the swing equation numerically
- To determine whether the system remains stable after a disturbance
- To plot rotor angle (δ) vs time (t)

Software Required

- Calculator / Computer
- MATLAB / Excel (optional for calculations)
- Given system parameters (P_m , P_e , H , frequency, etc.)

Theory

Transient Stability

Transient stability refers to the ability of a power system to remain in synchronism after a **large disturbance** such as:

- Faults
- Sudden load changes
- Loss of generation

Swing Equation

The basic equation governing rotor

dynamics is: [

$$\frac{d^2 \delta}{dt^2} = \frac{P_m - P_e}{M}$$

]

Where:

- δ = rotor angle (electrical degrees)
- P_m = mechanical input power
- P_e = electrical output power
- M = inertia constant

Output

Observation Table**Time (t) δ (rad) P_e (pu) $P_m - P_e$ Acceleration ω δ (next)**

0	0.2	...	...	...	0	...
0.01	...	...	...	...	...	...

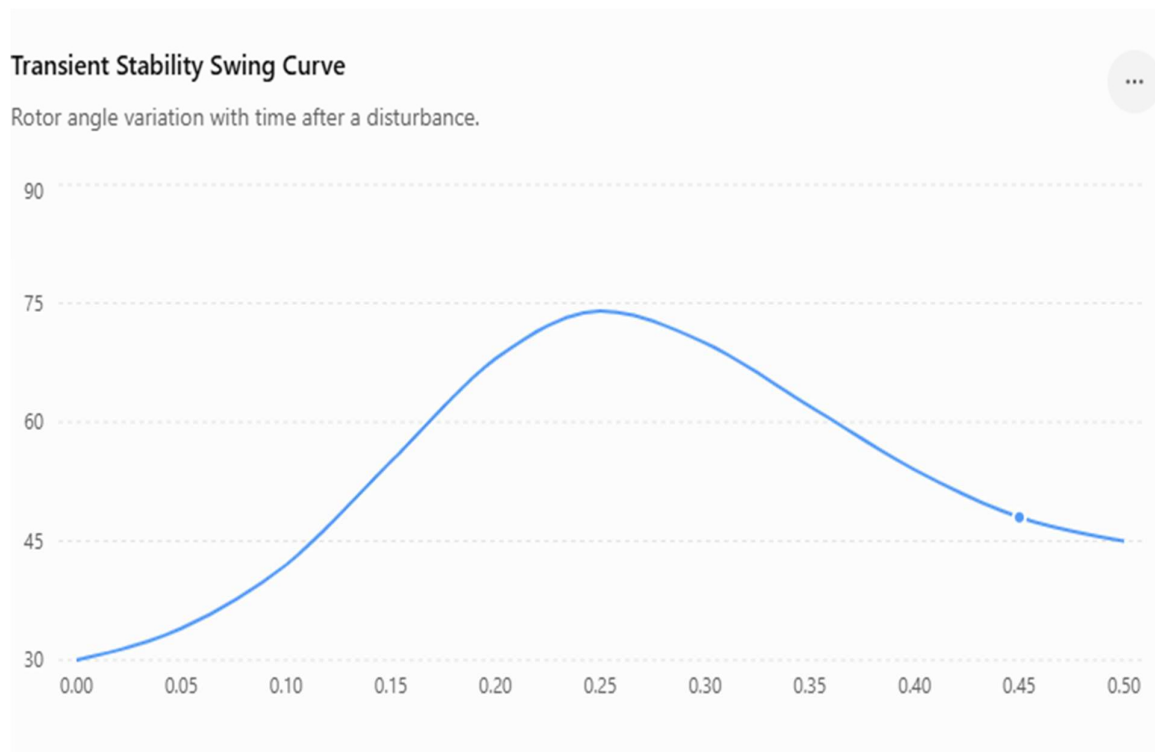
Calculations

- Compute $P_e = P_{\max} \sin \delta$
- Substitute values step-by-step
- Continue until sufficient time

Graph

Plot:

- **Rotor Angle (δ) vs Time (t)**



Point-by-Point Method

This is a **numerical method** used to solve the swing equation step-by-step.

Key equations:

1. **Acceleration:**

$$P_a = P_m - P_e$$

2. **Angular velocity:**

$$\omega_{n+1} = \omega_n + \alpha_n \Delta t$$

3. **Rotor angle:**

$$\delta_{n+1} = \delta_n + \omega_{n+1} \Delta t$$

Procedure

1. Assume initial conditions:
 - δ_0 (initial angle)
 - $\omega_0 = 0$
2. Choose a small time step (Δt), e.g., 0.01 sec
3. For each time step:
 - Calculate electrical power P_e
 - Compute accelerating power ($P_m - P_e$)
 - Calculate acceleration
 - Update angular velocity
 - Update rotor angle
4. Repeat calculations step-by-step
5. Observe whether δ increases indefinitely or oscillates

Sample Data (Example)

- $P_m = 0.8$ pu
- $P_e = P_{\max} \sin \delta$
- $P_{\max} = 1.2$ pu
- $H = 5$ MJ/MVA
- $f = 50$ Hz
- $\Delta t = 0.01$ sec

Interpret:

- If δ oscillates $\rightarrow$ System is **stable**
- If δ increases continuously $\rightarrow$ System is **unstable**

Precautions

- Use small time steps for accuracy
- Maintain consistent units
- Avoid rounding errors

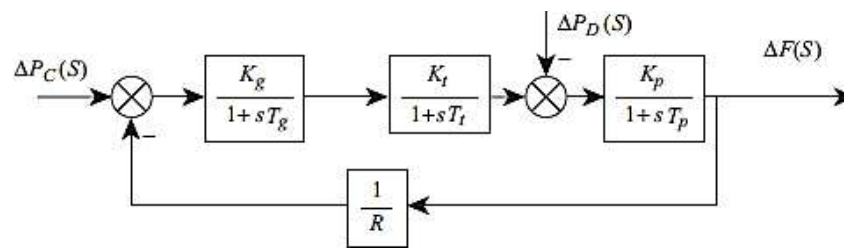
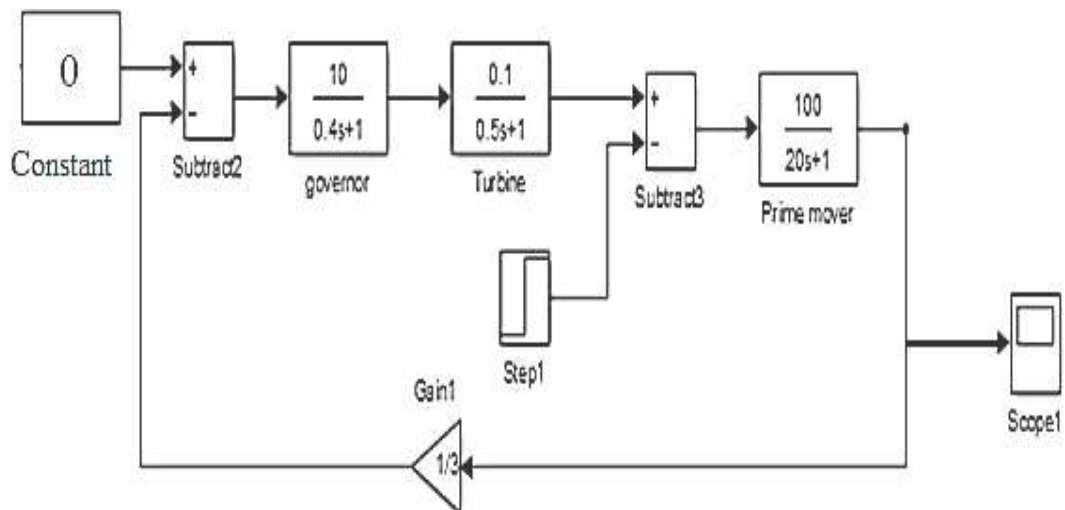
Result



Viva Questions

1. What is transient stability?
2. What is the swing equation?
3. Why is the point-by-point method used?
4. What happens during instability?
5. Define critical clearing angle

LOAD FREQUENCY CONTROL OF SINGLE AREA CONTROLLER

Block Diagram**Simulation Diagram**

11(a).LOAD FREQUENCY CONTROL OF SINGLE AREA CONTROLLER

Aim:- To study the Single Area load frequency control without PI controller.

Software Required: Matlab Software.

Formula Used

$$G_G = K_G / (1 + ST_G) \quad G_T = K_T / (1 + ST_T) \quad G_P = K_P / (1 + ST_P)$$

$$K_P = 1/D \quad \text{Where } D = \text{damping coefficient}$$

G_G = Transfer function of governor

G_T = Transfer function of turbine

G_P = Transfer function of Power system

K_P = gain of power system

K_t = gain of turbine

K_g = Gain of governor

T_p = Time constant of power

system T_g = Time constant of

Governor T_t = Time constant of

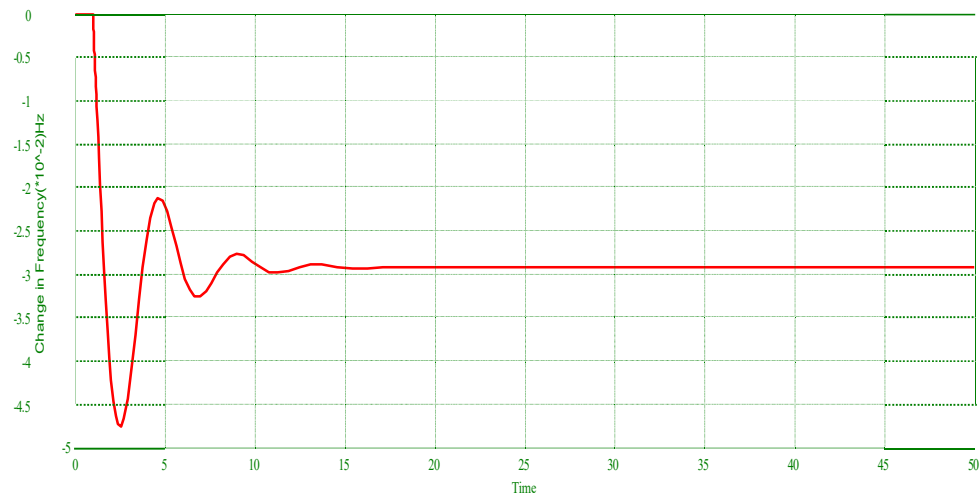
Turbine

R = Speed regulation of Governor.

Procedure

1. Open Matlab Simulink File New Model.
2. Open Simulink Library And Browse The Components.
3. Connect The Components As Per Circuit Diagram.
4. Set The Desired Voltage And Required Frequency.
5. Simulate The Circuit Using Simulink.
6. Plot The Waveforms.

Output Model Graph



Single Area Load Frequency Control without PI control

Result

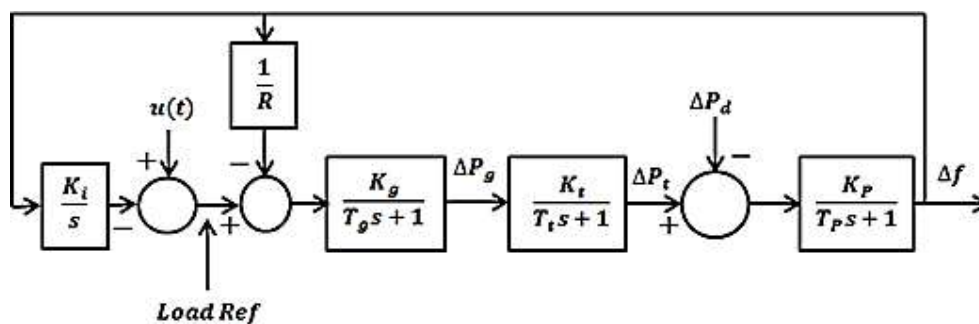
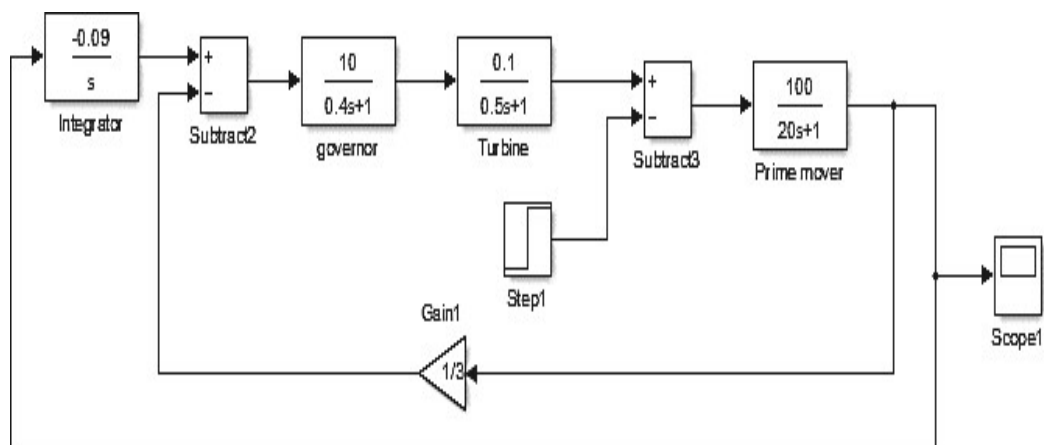


Viva Questions:

1. What is the function of load frequency control?
2. What are the methods of load frequency control?
3. What is load frequency mechanism?
4. What is dynamic response?
5. Why steady state error is not zero in single area system without control?



LOAD FREQUENCY CONTROL OF TWO AREA CONTROLLER

Block Diagram**Simulation Diagram**

11(b). LOAD FREQUENCY CONTROL OF TWO AREA CONTROLLER

Aim:- To study the load frequency control with PI controller.

Software Required Matlab Software.

Formula Used

$$G_G = K_G / (1 + S T_G) \quad G_T = K_T / (1 + S T_T) \quad G_P = K_P / (1 + S T_P)$$

$$K_P = 1/D \quad \text{Where } D = \text{damping coefficient}$$

G_G = Transfer function of governor

G_T = Transfer function of turbine

G_P = Transfer function of Power system

K_P = gain of power system

K_T = gain of turbine

K_G = Gain of governor

T_P = Time constant of power

system T_G = Time constant of

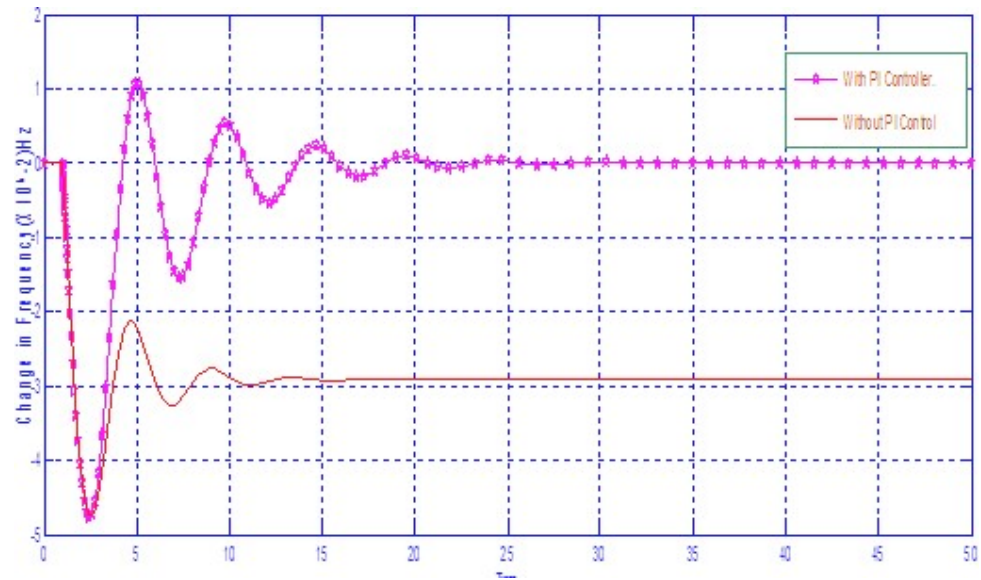
Governor T_T = Time constant of

Turbine

R = Speed regulation of Governor.

Procedure

1. Open Matlab Simulink File New Model.
2. Open Simulink Library And Browse The Components.
3. Connect The Components As Per Circuit Diagram.
4. Set The Desired Voltage And Required Frequency.
5. Simulate The Circuit Using Simulink.
6. Plot The Waveforms.

Output Model Graph

Single Area Load Frequency Control with and without PI control

Result

Viva Questions:

1. What is area control error?
2. Why negative feedback is given for integral controller?
3. How can we improve the dynamic response of a single area system with controller in terms of speed of response?
4. What is the function of load frequency control?
5. What are the methods of load frequency control?